

# Project 2: User Programs

## Preliminaries

Fill in your name and email address.

钟骏宇 [2400012946@stu.pku.edu.cn](mailto:2400012946@stu.pku.edu.cn)

If you have any preliminary comments on your submission, notes for the TAs, please give them here.

Please cite any offline or online sources you consulted while preparing your submission, other than the Pintos documentation, course text, lecture notes, and course staff.

Claude Sonnet/Opus 4.6 for code understanding and debug

## Argument Passing

### DATA STRUCTURES

A1: Copy here the declaration of each new or changed struct or struct member, global or static variable, typedef, or enumeration. Identify the purpose of each in 25 words or less.

Nothing.

### ALGORITHMS

A2: Briefly describe how you implemented argument parsing. How do you arrange for the elements of `argv[]` to be in the right order?  
How do you avoid overflowing the stack page?

首先使用 `strtok_r()` 解析命令行字符串，将每个参数的指针存储在局部的 `argv[64]` 数组中。然后，我们逆序将参数字符串压入用户栈，并将每个字符串压栈后的实际地址（对应的 `esp`）记录在 `arg_addr[]` 数组中。

对 `esp` 进行字节对齐之后插入 `NULL` 哨兵。接着再次逆序压入保存在 `arg_addr[]` 中的地址。再依次压入 `argv`（指向 `arg_addr[0]` 的指针）、`argc` 以及一个伪造的返回地址 `NULL`。

避免栈页溢出：用户栈只有一页（4096 字节），栈底为 `stack_bottom = PHYS_BASE - PGSIZE`。代码在两处进行边界检查：

1. 压字符串时：每次 `*esp -= len` 之后立即判断 `*esp < stack_bottom`，若越界则释放临时页并返回 `false`。
2. 压指针/argc/返回地址之前：计算这部分所需的总字节数 `remaining = (argc+1)*sizeof(char*) + sizeof(char**) + sizeof(int) + sizeof(void*)`，pre-flight 检查 `(uintptr_t)*esp - remaining < (uintptr_t)stack_bottom`，若空间不足同样提前返回 `false`。

## RATIONALE

A3: Why does Pintos implement `strtok_r()` but not `strtok()`?

`r` 代表线程可重入，根据 C 标准库的定义，`strtok()` 利用全局静态 `char` 指针去保存字符串，不安全。这里 `strtok_r()` 通过从传入的 `save_ptr` 保存状态来避免全局静态变量，安全。

A4: In Pintos, the kernel separates commands into a executable name and arguments. In Unix-like systems, the shell does this separation. Identify at least two advantages of the Unix approach.

1. 使得内核更简洁，可维护性更好
2. 更加安全稳定，shell 解析命令行时出错例如参数越界最多只会 segmentation fault，而内核来做可能直接 kernel panic 了，破坏性更大。
3. 命令行解析交给 shell 后，只要 kernel api 支持，非官方的 shell 就可以设计各种各样语法和功能，把任务交给社区。也方便实现例如环境变量，通配符等功能。

## System Calls

### DATA STRUCTURES

B1: Copy here the declaration of each new or changed struct or struct member, global or static variable, typedef, or enumeration. Identify the purpose of each in 25 words or less.

In `userprog/process.h` —— `child_info`，用来实现父子进程之间的通信和同步。

```
struct child_info {
    tid_t tid;                /* 子线程的 TID */
    int exit_status;          /* 子进程死亡前设置的退出状态码 */
    bool exited;             /* 子进程调用 process_exit() 后设为 true */
    bool waited;             /* 父进程对该子进程调用 process_wait() 后设为 true */
    struct semaphore sema;    /* 父进程在此阻塞等待子进程退出（初始为 0） */
    struct semaphore load_sema; /* 父进程在此阻塞等待子进程 load() 结束（初始为 0） */
    bool load_success;        /* 记录子进程的 ELF 文件是否成功加载 */
    int ref_count;            /* 共享所有权引用计数；降为 0 时释放内存 */
    struct list_elem elem;    /* 用于插入父进程 children 列表的链表节点 */
};
```

\*\*In `threads/thread.h`，父子进程相关和读写相关。

```
int exit_status;            /* 报告给父进程的退出状态码；默认为 0 */
struct list children;       /* 该线程生成的所有子进程的 child_info 列表 */
struct child_info *child_info; /* 指向该线程在父进程列表中的专属 child_info 的指针 */
struct file *exec_file;     /* 指向当前正在运行的可执行文件的句柄（用于禁止写入） */
struct file *fd_table[128]; /* 进程私有的文件描述符表；0 和 1 保留 */
int fd_next;                /* 下一个可分配的 fd；从 2 开始 */
```

\*\*In `userprog/syscall.c`

```
static struct lock filesys_lock; /* 全局文件系统锁，确保文件系统调用的原子性 */
```

B2: Describe how file descriptors are associated with open files. Are file descriptors unique within the entire OS or just within a single process?

每个线程维护一个 struct file \* 指针数组 fd\_table[128]。

当执行打开文件操作 (SYS\_OPEN) 时, filesys\_open() 会返回一个新的 struct file \*; 系统将其存储在 fd\_table[fd\_next++] 中。

文件描述符仅在单个进程内部唯一。

## ALGORITHMS

B3: Describe your code for reading and writing user data from the kernel.

### 1. 检查指针合法性

首先为了保证内核的安全性, 我们设计了三个辅助函数来检查。

- check\_ptr(ptr): 如果指针为 NULL、位于 PHYS\_BASE 之上, 或者在当前进程的页目录中未映射, 则拒绝该指针, 并直接调用 syscall\_exit(-1)。否则返回成功。
- check\_buf(buf, size): 从第一个页边界起逐页检查调用 check\_ptr。
- check\_str(s): 逐字节遍历字符串, 逐页调用 check\_ptr, 直到遇到 '\0' 则返回成功。(多了检查字符串格式的功能)

于是可以分别用来检查普通指针, 缓冲区和字符串指针, 保证后续访问安全。

### 2. 读取用户数据 (SYS\_READ)

在验证了 buffer 的合法性后:

如果 fd == STDIN\_FILENO (标准输入), 内核会通过循环调用 input\_getc(), 将键盘缓冲区的字符逐个写入用户提供的 buffer 中。

如果是普通文件

- 获取 filesys\_lock
- 通过 thread\_current()->fd\_table[fd] 找到对应的 struct file \*
- 然后调用 file\_read()
- 释放

### 3. 写入用户数据 (SYS\_WRITE)

同理, 在验证了 buffer 的安全后:

如果 fd == STDOUT\_FILENO (标准输出), 内核会直接调用 putbuf(buffer, size), 将用户内存中的数据打印到控制台。

如果是普通文件

- 获取 filesys\_lock
- 找到对应的 struct file \*
- 调用 file\_write()
- 释放

当前设计能很好的保护内存安全和文件系统的并发安全。

B4: Suppose a system call causes a full page (4,096 bytes) of data to be copied from user space into the kernel. What is the least and the greatest possible number of inspections of the page table (e.g. calls to pagedir\_get\_page()) that might result? What about for a system call that only copies 2 bytes of data? Is there room for improvement in these numbers, and how much?

当前实现 (check\_buf) 会按照跨页数量进行检查:

4096: 最少 1 次, 最多 2 次, 分别对应刚好在一页和跨页的情况。

2: 最少 1 次, 最多 2 次, 分别对应刚好在一页和跨页的情况。

改进: 使用 pintos 手册里的方法 2 应该可以不用显式调用 `pagedir_get_page` 来检查每个页, 调用次数降为 0。

B5: Briefly describe your implementation of the "wait" system call and how it interacts with process termination.

`SYS_WAIT` 在底层调用 `process_wait(child_tid)`, 我们直接看 `process_wait` 的实现:

`process_wait` 会遍历调用者的 `children` 列表, 这个 `children` 列表里存储着由父子进程共享的 `child_info` 结构体。

尝试寻找 `tid` 匹配的 `child_info` 结构体。如果未找到, 或者 `ci->waited` 已经被标记为 `true` (防止重复 wait), 则立即返回 -1。否则, 它将 `ci->waited` 设为 `true`, 并调用 `sema_down(&ci->sema)` 阻塞当前进程。

当子进程最终调用 `thread_exit()` 进而触发 `process_exit()` 时, 它会将自己的 `exit_status` 写入 `ci->exit_status`, 将 `ci->exited` 设为 `true`, 并调用 `sema_up(&ci->sema)`——唤醒沉睡的父进程。父进程醒来后读取状态码, 将 `child_info` 从列表中移除, 递减 `ref_count`, 如果 `count=0` 则释放该结构体。

如果子进程在父进程调用 `wait` 之前就已经退出了, 那么信号量的值已经变为 1, 此时 `sema_down` 会立刻通过而不会阻塞, 父进程可以直接读取到预存的退出状态。

B6: Any access to user program memory at a user-specified address can fail due to a bad pointer value. Such accesses must cause the process to be terminated. System calls are fraught with such accesses, e.g. a "write" system call requires reading the system call number from the user stack, then each of the call's three arguments, then an arbitrary amount of user memory, and any of these can fail at any point. This poses a design and error-handling problem: how do you best avoid obscuring the primary function of code in a morass of error-handling? Furthermore, when an error is detected, how do you ensure that all temporarily allocated resources (locks, buffers, etc.) are freed? In a few paragraphs, describe the strategy or strategies you adopted for managing these issues. Give an example.

设计辅助函数来处理指针检查和报错就行

指针的验证都委托给了 `check_ptr()`, `check_buf()` 和 `check_str()`。一旦验证失败, 这些辅助函数会直接调用 `syscall_exit(-1)` 进而触发 `thread_exit()`。由于 `thread_exit()` 必定会调用 `process_exit()`, 自然进行资源清理。

以 `SYS_WRITE` 为例:

- `get_arg(f, 0/1/2)` 会验证 `esp+4`、`esp+8`、`esp+12` (三个参数所在的位置)。遇到任何非法地址, 立刻进入 `syscall_exit(-1)`。
- `check_buf(buf, size)` 逐页验证用户缓冲区: 先检查起始地址, 再按 `PGSIZE` 步进逐一检查每个页边界, 最后检查末字节, 确保跨越多页的缓冲区中间页不被漏检。
- 如果上述验证全部通过, 才会调用 `lock_acquire(&filesys_lock)`, 执行写操作, 最后释放锁。

如果步骤 1 或 2 失败, 程序会在获取任何锁之前就进入 `syscall_exit(-1)`, 因此在进程销毁的过程中不持有任何锁。我们始终确保在所有内存验证才通过。

## SYNCHRONIZATION

B7: The "exec" system call returns -1 if loading the new executable fails, so it cannot return before the new executable has completed loading. How does your code ensure this? How is the load success/failure status passed back to the thread that calls "exec"?

process\_execute() 在堆上分配一个 child\_info 结构体并且在父子进程间，将其中的 load\_sema 信号量初始化为 0，启动子线程。

在 thread\_create() 返回后，父进程会立即调用 sema\_down(&ci->load\_sema) 将自己阻塞。此时子进程执行 load() 操作，将加载的布尔结果写入 ci->load\_success，最后调用 sema\_up(&ci->load\_sema)。父进程才能被唤醒去 return。

B8: Consider parent process P with child process C. How do you ensure proper synchronization and avoid race conditions when P calls wait(C) before C exits? After C exits? How do you ensure that all resources are freed in each case? How about when P terminates without waiting, before C exits? After C exits? Are there any special cases?

我们通过在堆上分配共享的 child\_info 结构体，并利用初始值为 2 的 ref\_count（引用计数）机制来解决所有竞争条件。父子双方都持有指针，每退出一方就将计数减 1，最后一个离开的一方负责释放内存。同步则通过初始值为 0 的信号量 sema 来实现。

- P 在 C 退出前调用 wait: P 调用 sema\_down 并被阻塞。C 退出时，写入状态码，调用 sema\_up 唤醒 P。P 醒来读取状态，递减 ref\_count（此时为 1，不释放内存），并将 ci 从列表中移除。由于 C 退出时已经递减过一次，最终 P 递减时发现计数归零，由 P 负责 free。
- P 在 C 退出后调用 wait: C 退出时已经调用了 sema\_up，并将 ref\_count 减至 1。P 随后调用 sema\_down 时会立刻返回（不阻塞）。P 读取预存的状态码，递减 ref\_count 至 0 并负责释放。
- P 未执行 wait 就提前退出，发生在 C 退出前: P 在执行 process\_exit 时，会遍历自己的 children 列表，将所有子进程的 ref\_count 从 2 减为 1。此时 C 仍在运行，且依然持有有效指针。当 C 最终退出时，它递减 ref\_count（1 变为 0），由 C 自己负责释放结构体。
- P 未执行 wait 退出，发生在 C 退出后: C 退出时已将 ref\_count 减至 1。P 退出时清理 children 列表，将其减至 0，由 P 负责释放。

特殊情况: waited = true 标志位确保了父进程无法对同一个子进程执行两次 wait 操作。第二次调用会直接返回 -1，从而避免了重复释放带来的内核崩溃问题。

## RATIONALE

B9: Why did you choose to implement access to user memory from the kernel in the way that you did?

我选择了第一种方法 "is to verify the validity of a user-provided pointer, then dereference it."

优点

- 逻辑清晰: 所有非法的指针故障都在一个明确的时间点被拦截，并且此时尚未修改任何内核状态。
- 降低复杂性: 无需在 exception.c 的缺页异常处理程序中编写复杂的恢复程序。
- 对于 Lab 2 而言，性能差异微乎其微；每个系统调用额外执行两次 pagedir\_get\_page 带来的延迟是完全可以忽略不计的。

B10: What advantages or disadvantages can you see to your design for file descriptors?

优点:

- $O(1)$  的时间复杂度：直接通过数组索引完成。
- 实现极其简单：每个 fd 都不需要额外的动态内存分配。
- 进程级别的隔离：一个进程的文件描述符表操作完全不会影响到另一个进程。

缺点：

- 容量受限：每个进程被硬编码限制最多只能同时打开 126 个文件。
- 无回收机制：当前实现中 fd\_next 只是单调递增，不会复用已被关闭的低位空闲槽位。
- 空间浪费：无论进程是否需要操作文件，struct thread 中相当于始终预留了 512 字节的空间。

B11: The default tid\_t to pid\_t mapping is the identity mapping.

If you changed it, what advantages are there to your approach?

我没改，毕竟 pintos 一个线程只包含一个进程。